

CHAPTER: SYSTEM DESIGN, ANALYSIS, AND GOVERNANCE

Project: Asset-Defender | Generated Q&A Chapter for Viva / Report Submission

Table of Contents

- 1. Project Workflow and Methodology
- 2. Modules, Models, and Data Pipeline
- 3. Scoring, Formulas, and Final Risk Fusion
- 4. Scraping, Extension Integration, and Storage
- 5. Security, Privacy, Legal Scope, and Admin Control
- 6. Validation, Performance, Operations, and Roadmap
- Appendix A: Consolidated Question–Answer Set (43 Questions)

1. Project Workflow and Methodology

Q1. What is the workflow of the project?

User authentication -> extension popup triggers analysis -> content script scrapes Instagram profile/message context -> client-side heuristics produce preliminary scores -> backend performs deep analysis + ML inference -> results are persisted in SQLite -> dashboard/admin pages visualize outcomes.

Q2. What is the literature survey and where is it located?

A dedicated literature-survey document is not currently present in the repository. The nearest technical references are model training scripts and model performance UI components.

Q3. What methodology is followed?

Hybrid methodology combining deterministic heuristics, supervised ML predictions, evidence-based confidence gating, and final decision fusion with risk-floor overrides for explicit phishing/scam indicators.

2. Modules, Models, and Data Pipeline

Q4. How many modules are used?

Major logical modules: authentication, extension messaging, Instagram scraping, client heuristics engine, server deep-analysis API, ML training/inference, persistence layer, and admin analytics/monitoring.

Q5. How many ML/NLP models are used and why?

Classifiers: 6 total (Profile: RF, XGBoost, GradientBoosting, LogisticRegression; Message: MultinomialNB, LogisticRegression) + 1 TF-IDF vectorizer. Primary/backup strategy provides reliability if one model fails to load or infer.

Q6. Which datasets are used for training?

Profile training dataset: profiledataset/train.csv

Message training dataset: messagedatasets/spam.csv

Q7. What is the ground-truth labeling process?

Current implementation relies on labels already present in these CSV datasets. A separate in-project manual labeling protocol and quality audit workflow are not yet implemented.

3. Scoring, Formulas, and Final Risk Fusion

Q8. How are client-side heuristic scores and preliminary risk computed?

Profile trust dimensions (structural/content/behavioral/photo) are computed, weighted, and adjusted by confidence/evidence constraints. Preliminary risk is derived from inverse trust plus uncertainty/negative-signal penalties and then clamped.

Q9. How is client-side message analysis computed?

Client extracts keyword, suspicious link/domain, repetition, burst, and timeline signals. It computes spamScore, urgentScore, burstScore, then combines them into overallRisk with weighted aggregation and scam-floor constraints.

Q10. How is server-side message analysis computed?

Server computes threatScore from lexical risk, phishing/credential cues, domain risk, repetition, and timing features; applies floor logic when scam signals are dense; then merges with ML message risk using max(heuristic, ML).

Q11. How is final risk score calculated?

Extension fusion currently emphasizes message dominance: $\text{finalRisk} = 0.3 * \text{profileRisk} + 0.7 * \text{messageRisk}$ (when both are available). Additional overrides can force scam/hacker if phishing evidence is explicit.

Q12. What rules are used for client and server calculations?

Client: trust and evidence gates, ratio behavior gates, explicit scam penalties, institutional safeguards. Server: weighted threat equations, classification-tag rules, profile calibration, confidence scoring, and insufficient-data constraints.

Q13. How are false positives vs false negatives managed?

By thresholding, confidence/evidence gating, suppression to insufficient-data when evidence is weak, and high-priority message phishing overrides to reduce dangerous false negatives for scam/hacker outputs.

Q14. What threshold tuning strategy is used?

Thresholds are currently rule-defined/manual and operationally tuned (e.g., medium/high bands and floor thresholds). Automated threshold optimization with validation curves is not yet in the pipeline.

4. Scraping, Extension Integration, and Storage

Q15. How is Instagram data scraped?

Content scripts parse DOM + embedded JSON/API responses on instagram.com pages, including profile stats, media metadata, and DM message text/timestamps where available.

Q16. How is extension layer connected and maintained?

Manifest V3 uses popup, background service worker, and content scripts with chrome.runtime/chrome.tabs messaging and chrome.storage.local persistence. Pending hidden-tab analyses are tracked and cleaned up in background logic.

Q17. Which files implement client prediction and database storage?

Client prediction and UI explanation: client/public/popup.js

Scraping: client/public/instagram_analyzer.js

Server DB schema/writes: server/database.js

Main DB file: server/database/app.db

Q18. How many databases are used and is project multi-user?

Primary production path uses one SQLite DB (app.db). A second ML-side SQLite file path exists in Python helper scripts. Multi-user support exists through JWT user identity + user_id-scoped records.

5. Security, Privacy, Legal Scope, and Admin Control

Q19. What security controls exist?

Implemented: bcrypt password hashing, JWT with expiry, role-based admin checks, CORS.

Not fully implemented: secret rotation policy, strict password policy enforcement, request rate limiting, CSRF/security-header hardening, complete audit pipeline.

Q20. What is privacy/data retention policy?

Data storage is implemented, but formal retention/deletion policy automation and explicit privacy policy documents are not present in current codebase.

Q21. What is legal/compliance scope and scraping legality status?

The repository does not include a finalized legal compliance framework. Compliance depends on platform ToS, consent model, and jurisdictional law. Legal review is required before production-scale deployment.

Q22. How can admin control all users' data?

Admin-specific JWT endpoints allow centralized user and analysis visibility (users list, analyses list, risk stats) and export capabilities from admin dashboard.

Q23. How is multi-tenant isolation handled?

Logical isolation is implemented via JWT userId and query filtering by user_id. Physical tenant isolation is not implemented because storage is shared SQLite.

6. Validation, Performance, Operations, and Roadmap

Q24. What model evaluation metrics are used beyond accuracy?

Current scripts report accuracy. Precision, recall, F1, ROC-AUC, confusion matrix reporting should be added for robust evaluation.

Q25. How do you test end-to-end and what coverage exists?

No complete first-party unit/integration/E2E test suite is currently evident in app code. Formal automated testing and coverage gates are recommended.

Q26. What is performance profile (latency/concurrency)?

No formal benchmark report is available in repository. Load testing is required to establish p50/p95 latency and max safe concurrency.

Q27. How are error handling and observability managed?

Try/catch and console logs are used; a Winston logger module exists. Central alerting/monitoring (Sentry/metrics dashboard/on-call alerts) is not fully integrated in current path.

Q28. What is deployment architecture (dev/staging/prod, CI/CD)?

Current setup is primarily local development (localhost services, local DB, extension host permissions). A fully defined staged CI/CD production architecture is not yet present.

Q29. What explainability output is shown to users?

UI presents score breakdowns, risk classes, confidence, evidence quality, and top reasons for profile/message/final labels.

Q30. What is human-in-the-loop flow for disputed predictions?

Admin can inspect analyses, but no formal dispute queue, reviewer assignment, override lifecycle, or feedback-to-retraining loop is implemented yet.

Q31. What abuse/adversarial scenarios are considered?

Covered partially: suspicious links, risky TLDs, impersonation terms, repetition bursts. Gaps remain for sophisticated obfuscation, homoglyph URLs, and multilingual adversarial text.

Q32. What backup/recovery plan exists?

No explicit automated backup/restore scripts are present for database and model artifacts. This should be formalized.

Q33. How is model drift/retraining handled?

No scheduled drift monitoring/retraining pipeline exists; retraining is manual or triggered if model files are missing.

Q34. What is model/rule versioning and rollback strategy?

Primary/backup model failover exists, but no full model registry/version tagging and rollback governance workflow is implemented.

Q35. What is roadmap?

Recommended roadmap: multilingual support, expanded evaluation metrics, automated threshold tuning, model registry + rollback, stronger API security/rate limits, full test automation, CI/CD, and policy/legal documentation.

Appendix A: Direct Q&A Consolidation

This chapter consolidates your original project-defense questions and additional governance/engineering questions into one submission-ready narrative format.